

INT102

Algorithmic Foundations & Problem Solving

Complete Exam Guide / 历年真题高频考点全解析

XJTLU Year 2 | Final / Resit 2020 - 2024 | CW1 + CW2

适用试卷: 2020-21 / 2021-22 / 2022-23 Final & Resit, 2023-24 Part2, CW1, CW2

All topics include: English explanation + Chinese annotation

以下内容: 英文讲解 + 中文注释, 选择题和大题分开预测

Exam Prediction (Based on Past 6 Papers)

The following prediction is derived from 6 past papers (2020-21 to 2022-23 Final+Resit) plus CW1 and CW2. Frequencies are shown as N/6 (or N/8 including coursework).

以下预测基于6套历年试卷 (2020-21至2022-23

Final+Resit) 以及CW1和CW2。考频以N/6 (含课程作业为N/8) 表示。

Part A (MCQ) - High Probability Topics:

选择题 (Part A) —— 高概率考点:

Rank	Topic	Frequency	Prediction
1 (MUST)	Time Complexity (Big-O)	6/6	3-4 questions GUARANTEED
2 (MUST)	BFS / DFS traversal order	5/6	1-2 questions GUARANTEED
3 (MUST)	MST: Kruskal edge order / Prim vertex order	4/6	1-2 questions very likely
4 (MUST)	Dijkstra shortest path labels	4/6	1-2 questions very likely
5 (MUST)	Sorting: comparisons & swaps (Bubble Selection)	4/6	1-2 questions very likely
6 (LIKELY)	P vs NP vs NP-Complete statements	3/6	1 question likely
7 (LIKELY)	LCS table filling	2/6	1 question possible

8 (LIKELY)	0/1 Knapsack $V[i,j]$ table	3/6	1 question possible
9 (LOW)	Floyd / Horspool / Coin Row	1-2/6	Low chance, know basics

Part B (Written) - High Probability Topics:

大题 (Part B) —— 高概率考点:

Rank	Topic	Frequency	Prediction
1 (MUST)	Divide and Conquer: algorithm + recurrence	4/6	20-27 marks GUARANTEED
2 (MUST)	0/1 Knapsack DP: table + optimal subset	3/6	10-15 marks very likely
3 (MUST)	P/NP/NPC: definitions + reduction + proof	3/6	12-18 marks very likely
4 (LIKELY)	MST: Prim + Kruskal + Dijkstra full steps	1/6	10-15 marks possible
5 (LIKELY)	LCS: table + backtrack	2/6	8-12 marks possible
6 (NEW)	Sequence Alignment (CW2 2023-24)	2/6	New! 15-30 marks possible
7 (LOW)	Coin Row / Change-making DP	1-2/6	Low chance

Key Prediction Insight (read carefully!):

[1] Divide & Conquer written question appears in 4/6 papers -> MUST prepare full template.

分治法大题在4/6套试卷中出现 → 必须准备完整答题模板。

[2] If you can ONLY prepare 3 topics, choose: (1) Divide & Conquer, (2) 0/1 Knapsack DP, (3) P/NP/NPC.

如果只能准备3个大题，选：（1）分治法，（2）0/1背包DP，（3）P/NP/NPC。

[3] Big-O MCQ appears in EVERY paper -> Memorise growth rate ranking (see Section 1).

Big-O选择题每套试卷都出现 → 背熟增长率排序（见第1节）。

[4] Sequence Alignment (Needleman-Wunsch / Smith-Waterman) appeared in CW2 2023-24 -> likely in Final 2024-25.

序列比对 (Needleman-Wunsch / Smith-Waterman) 在2023-24 CW2中出现 → 2024-25 Final很可能考。

Section 1: Time Complexity Analysis / 时间复杂度分析

Appears in ALL 6/6 papers (3-4 MCQs per paper). This is the #1 highest-frequency topic.

所有6/6套试卷均出现（每套3-4道选择题）。这是考频最高的第1名。

Growth Rate Ranking (MEMORISE this order from slowest to fastest):

增长率排序（必须背熟，从最慢到最快）：

Order	Notation	Example Function
1 (slowest)	$O(1)$	$\text{sum} = n*(n+1)/2$
2	$O(\log \log n)$	$50 \log(\log n) + 20$
3	$O(\log n)$	$\text{while } x < n: x = x * 2$
4	$O(\sqrt{n})$	$\text{for } i \text{ in range(int(sqrt(n))): ...}$
5	$O(n)$	Linear search

6	$O(n \log n)$	Merge Sort: $T(n) = 2T(n/2) + n$
7	$O(n^2)$	Bubble Sort: double loop
8	$O(2^n)$	$T(n) = 2T(n-1) + 1$
9 (fastest)	$O(n!)$	Brute force TSP

Big-O Definition (MCQ trap!):

Big-O 定义 (选择题陷阱!) :

$f(n)$ is $O(g(n))$

if and only if:

there exist constant $c > 0$ and $n_0 > 0$

such that: $f(n) \leq c * g(n)$ for ALL $n > n_0$

MCQ WRONG answer: 'for all n , $f(n) \leq g(n)$ ' -> this is NOT the definition of $O()$.

选择题错误选项: '对所有 n 都有 $f(n) \leq g(n)$ ' → 这不是Big-O的定义, 不要选!

MCQ WRONG answer: ' $O(n*(n+1)/2)$ ' -> this is NOT simplified! Always simplify: $O(n^2)$.

选择题错误选项: ' $O(n*(n+1)/2)$ ' → 没有化简! 必须化简: $O(n^2)$ 。

Common Code Patterns -> Identify $O()$:

常见代码模式 → 判断时间复杂度:

Code Pattern	Time Complexity	Notes
$sum = n*(n+1)/2$	$O(1)$	TRAP! Not $O(n)$! Formula = constant time.
while $x < n$: $x = x * 2$	$O(\log n)$	x doubles each iteration.
for i in range(n): (single loop)	$O(n)$	
for i in range(n): for j in range(n):	$O(n^2)$	
$T(n) = 2T(n/2) + n$	$O(n \log n)$	Merge Sort recurrence.
$T(n) = T(n/2) + 1$	$O(\log n)$	Binary Search recurrence.
$T(n) = T(n-1) + n$	$O(n^2)$	Worst-case Insertion Sort.

Section 2: Graph Traversal - BFS & DFS / 图遍历: BFS与DFS

Appears in 5/6 papers. Given adjacency matrix or list, write the exact vertex visit order. Ties resolved by alphabetical order.

5/6套试卷出现。给出邻接矩阵或邻接表, 写出顶点访问顺序。平局按字母序处理。

BFS (Breadth-First Search):

BFS (广度优先搜索) :

Rule: Use a QUEUE (FIFO). Visit all neighbours at current depth BEFORE going deeper. Each vertex is enqueued only ONCE (when first discovered).

规则: 使用队列 (FIFO)。在当前深度的所有邻居都访问完后, 才继续深入下一层。每个顶点只入队一次 (首次发现时)。

DFS (Depth-First Search):

DFS（深度优先搜索）：

Rule: Use a STACK (LIFO) or recursion. Go as deep as possible along one branch, then BACKTRACK.
Ties: pick smallest alphabetical unvisited neighbour.

规则：使用栈（LIFO）或递归。沿一条分支尽可能深入，然后回溯。平局：选字母序最小的未访问邻居。

BFS vs DFS - Key Differences:

BFS与DFS —— 关键区别：

	BFS	DFS
Data Structure	Queue (FIFO)	Stack / Recursion (LIFO)
Visit Order	Level by level	Deep branch first, then backtrack
Shortest Path (unweighted)	YES (first time visited)	NO
Space (worst case)	$O(W)$ where $W = \text{max width}$	$O(H)$ where $H = \text{max depth}$
Use Case	Shortest path in unweighted graph	Topological sort, SCC

EXAM TIP: Write the visit order VERTEX-BY-VERTEX (one per line). Do NOT write the whole sequence in one line - you may lose marks if you skip one by mistake.

考试技巧：逐顶点写出访问顺序（每行一个）。不要全部写在一行——如果跳过一个，可能会失分。

Section 3: Minimum Spanning Tree - Kruskal & Prim / 最小生成树

Appears in 4/6 papers for MCQs; also appears as a 'show-all-steps' question in Part B (2/6 papers).

4/6套试卷出选择题；2/6套试卷在大题中出现，要求写出完整步骤。

Kruskal's Algorithm:

Kruskal算法（按边权重从小到大选边）：

Step 1: Sort ALL edges by weight (ascending). Step 2: Add edges one by one. Step 3: SKIP any edge that creates a CYCLE. Step 4: Stop when exactly $|V|-1$ edges selected.

第1步：将所有边按权重从小到大排序。第2步：逐条加边。第3步：若加入后形成环，则跳过该边。第4步：选满 $|V|-1$ 条边时停止。

Prim's Algorithm:

Prim算法（从指定顶点开始，每次选最近的未访问顶点）：

Step 1: Start from a given vertex (e.g. 'vertex a is selected first'). Step 2: Maintain distance to ALL unreachable vertices. Step 3: At each step, select the UNVISITED vertex with SMALLEST distance. Step 4: Update distances of its neighbours.

第1步：从指定顶点开始（如'顶点a先被选中'）。第2步：维护到所有未访问顶点的距离。第3步：每步选距离最小的未访问顶点。第4步：更新其邻居的距离。

MST Properties - MCQ Traps (2/6 papers):

MST性质 —— 选择题陷阱（2/6套试卷）：

Statement	True / False	Reason
If e is a minimum-weight edge in G, it must be in SOME MST.	TRUE	MST must pick it; otherwise the tree would not be minimal.
If e is a minimum-weight edge in G, it must be in EVERY MST.	FALSE	Multiple edges have the same min weight, only one is needed.
If e is a maximum-weight edge in G, it must NOT be in any MST.	FALSE	It may be the ONLY edge connecting a disconnected component.
If all edge weights are distinct, the MST is UNIQUE.	TRUE	No tie-breaking needed.
If edge weights are NOT all distinct, there MUST be > 1 MST.	FALSE	Having ties does NOT guarantee multiple MSTs.
If all edge weights are equal, every spanning tree is an MST.	TRUE	All MSTs have the same total weight.

Section 4: Dijkstra's Algorithm / Dijkstra最短路径算法

Appears in 4/6 papers. Given a weighted graph and a source vertex, find the label (distance, predecessor) for every vertex after Dijkstra terminates.

4/6套试卷。给出加权图和源点，求Dijkstra结束后每个顶点的label（距离，前驱）。

Label format: vertex(distance, predecessor)

Label格式：顶点(距离，前驱)

Example: a(0,-) b(2,a) c(4,a) d(6,b) e(6,b)

示例：a(0,-) b(2,a) c(4,a) d(6,b) e(6,b)

CRITICAL: Dijkstra's tree is a SHORTEST-PATH TREE, NOT an MST! MST minimises TOTAL edge weight; Dijkstra minimises path weight from source.

关键：Dijkstra生成的树是最短路径树，不是MST！MST最小化总边权；Dijkstra最小化从源点出发的路径权重。

Dijkstra does NOT work with negative-weight edges. For negative weights, use Bellman-Ford.

Dijkstra不能处理负权边。对于负权边，使用Bellman-Ford算法。

Dijkstra Step-by-Step (exam writing format):

Dijkstra逐步求解（考试书写格式）：

Step	Visited Set	a	b	c	d	e	Selected This Step
Init	{}	0,-	inf,-	inf,-	inf,-	inf,-	a (dist=0)
1	{a}	-	2,a	4,a	inf,-	inf,-	b (dist=2)
2	{a,b}	-	-	4,a	9,a	inf,-	c (dist=4)
3	{a,b,c}	-	-	-	7,b	inf,-	d (dist=7)
4	{a,b,c,d}	-	-	-	-	11,c	e (dist=11)

Section 5: Sorting Algorithms - Bubble & Selection / 排序算法

Appears in 4/6 papers as MCQs. Given a concrete array, count the number of KEY COMPARISONS and SWAP operations.

4/6套试卷出选择题。给出具体数组，计算关键比较次数和交换次数。

Bubble Sort:

冒泡排序：

Property	Bubble Sort
Key Comparisons (any input)	$n(n-1)/2$ (FIXED, independent of input)
Best-case Swaps	0 (already sorted)
Worst-case Swaps	$n(n-1)/2$ (reverse sorted)
Time Complexity	$O(n^2)$

Selection Sort:

选择排序：

Property	Selection Sort
Key Comparisons (any input)	$n(n-1)/2$ (FIXED)
Swaps (any input)	$\leq n-1$ (at most one swap per outer loop iteration)
Time Complexity	$O(n^2)$

Worked Example (MCQ favourite):

计算示例（选择题最爱考）：

Array A[0..5] = [6, 1, 2, 3, 4, 5] (sort ascending) Bubble Sort comparisons = $5+4+3+2+1 = 15$ Bubble Sort swaps = $1+1+1+1+1 = 5$ (6 has to move past 5 smaller elements) Selection Sort comparisons = 15 (always the same) Selection Sort swaps = 1 (swap 6 with 1, then array is sorted!)

数组A[0..5] = [6, 1, 2, 3, 4, 5]（升序排序）冒泡排序比较次数 = $5+4+3+2+1 = 15$ 冒泡排序交换次数 = $1+1+1+1+1 = 5$ 选择排序比较次数 = 15（固定）选择排序交换次数 = 1（6与1交换后数组已有序！）

Section 6: Divide and Conquer / 分治法（大题必考）

THE most important Part B topic: appears in 4/6 papers as a 12-27 mark question. You MUST memorise the full answer template.

最重要的Part B题型：4/6套试卷出12-27分大题。必须背熟完整答题模板。

Standard Answer Template (memorise!):

标准答题模板（背下来！）：

Part 1: Describe Divide-and-Conquer Technique (3 marks) Step 1 (DIVIDE): Split the problem into smaller subproblems of the same type. Step 2 (CONQUER): Solve each subproblem recursively (or directly if small enough). Step 3 (COMBINE): Merge the solutions of the subproblems into the solution for the original problem.

第1部分：描述分治思想（3分）第1步（分解）：将问题分成若干个相同类型的小问题。

第2步（征服）：递归求解每个小问题（规模足够小时直接求解）。

第3步（合并）：将小问题的解合并为原问题的解。

Common Recurrence Patterns:

常见递推关系模式：

Algorithm	Recurrence T(n)	Solution
-----------	-----------------	----------

Binary Search	$T(n) = T(n/2) + 1$	$O(\log n)$
Merge Sort	$T(n) = 2T(n/2) + n$	$O(n \log n)$
Peak-finding (unimodal array)	$T(n) = T(n/2) + 1$	$O(\log n)$
Max-Min (divide-conquer)	$T(n) = 2T(n/2) + 2$	$O(n)$ (exact: $3n/2 - 2$ comparisons)
Strassen Matrix Mult.	$T(n) = 7T(n/2) + n^2$	$O(n^{2.81})$

Classic Exam Question - Peak-finding in unimodal array:

经典考题 —— 先增后减数组找峰值:

ALGORITHM FindPeak(A[l..r]):

if $l == r$ then return l

$m = \text{floor}((l + r) / 2)$

if $A[m] < A[m+1]$ then

return FindPeak(A[m+1..r]) // peak is to the RIGHT

else

return FindPeak(A[l..m]) // peak is to the LEFT (or at m)

Recurrence: $T(n) = T(n/2) + 1$

Solution: $T(n) = O(\log n)$ (same as Binary Search)

Exam asks: 'What is the output for $A[0..7] = [12,13,14,15,14,13,12,11]$?' Answer: returns position 3 (value 15, the peak). Comparisons needed: 3.

考试会问: ' $A[0..7] = [12,13,14,15,14,13,12,11]$ 的输出是什么? '

答: 返回位置3 (值15, 峰值)。需要的比较次数: 3次。

Section 7: Dynamic Programming - 0/1 Knapsack / 动态规划: 0/1背包

Appears in 3/6 papers (MCQ + Part B). Given items with weights and values, and a knapsack capacity W , fill the DP table $V[i,j]$, find optimal value, and identify the optimal subset by backtracking.

3/6套试卷 (选择题+大题)。给出物品重量和价值, 以及背包容量 W , 填写DP表格 $V[i,j]$, 求最优值, 并通过回溯找出最优子集。

Recurrence Relation (memorise!):

递推关系 (背下来!) :

$V[i, j] = 0$ if $i = 0$ or $j = 0$

$V[i, j] = V[i-1, j]$ if $j < w_i$ (item i does NOT fit)

$V[i, j] = \max\{ V[i-1, j], V[i-1, j-w_i] + v_i \}$ if $j \geq w_i$

Worked Example (exam standard format):

计算示例 (考试标准格式) :

i (item)	w _i	v _i	j=0	j=1	j=2	j=3
0 (base)	-	-	0	0	0	0
1	w ₁ =2	v ₁ =12	0	0	12	12
2	w ₂ =1	v ₂ =10	0	10	12	22
3	w ₃ =3	v ₃ =20	0	10	12	22
						V[3,3] = 22 (items 1+2)

Backtracking to Find Optimal Subset:

回溯找出最优子集:

Start at $V[n, W]$. If $V[i, j] \neq V[i-1, j]$ -> item i IS selected; then move to $V[i-1, j-w_i]$. Else -> item i is NOT selected; move to $V[i-1, j]$.

从 $V[n, W]$ 开始。如果 $V[i, j] \neq V[i-1, j]$ -> 物品 i 被选中; 然后跳到 $V[i-1, j-w_i]$ 。否则 -> 物品 i 未被选中; 跳到 $V[i-1, j]$ 。

Section 8: P / NP / NP-Complete Theory / P/NP/NPC理论

Appears in 3/6 papers (MCQ + Part B 12-18 marks). MCQs test recognition of P vs NP vs NP-Complete. Part B asks you to define P, NP, NPC; explain polynomial-time reduction; prove a problem is NPC.

3/6套试卷 (选择题+大题12-18分)。选择题考查P/NP/NPC的判断; 大题要求定义P、NP、NPC; 解释多项式时间归约; 证明某问题是NPC。

Definitions (memorise exactly):

定义 (精确背诵):

P (Polynomial time): The class of decision problems that can be SOLVED by a deterministic algorithm in worst-case polynomial time.

P类 (多项式时间): 可以在最坏情况下用确定性算法在多项式时间内【求解】的决策问题类。

NP (Nondeterministic Polynomial time): The class of decision problems for which a given YES-certificate can be VERIFIED in polynomial time.

NP类: 可以在多项式时间内【验证】给定YES证书的决策问题类。

NP-Complete (NPC): A problem that (1) belongs to NP, and (2) every problem in NP can be polynomially reduced TO it. If any NPC problem is in P, then $P = NP$.

NP完全 (NPC): 满足(1)属于NP, 且(2)所有NP问题都可以多项式归约到它的问题。如果任何一个NPC问题属于P, 则 $P=NP$ 。

MCQ True/False - Common Statement Set:

选择题判断 —— 常见陈述组合:

Statement	True / False	Explanation
P problems can be solved in worst-case polynomial time.	TRUE	
NP problems can be solved in non-polynomial time.	FALSE	Misleading! NP = polynomial VERIFICATION, not non-p
NP problems can be verified in polynomial time.	TRUE	

Finding MST is a P problem.	TRUE	Prim/Kruskal run in $O(E \log V)$.
0/1 Knapsack is an NP-Complete problem.	TRUE	But the decision version is NPC.
Vertex Cover is an NP-Complete problem.	TRUE	
Travelling Salesman (TSP) is an NP-Complete problem.	TRUE	
Dijkstra's tree is an MST.	FALSE	It is a shortest-path tree, NOT MST.

How to Prove a Problem is NP-Complete (Part B template):

如何证明一个问题是NPC（大题模板）：

Step 1: Prove the problem belongs to NP (give a polynomial-time verifier). Step 2: Choose a known NPC problem (e.g. 3-SAT, Vertex Cover). Step 3: Construct a polynomial-time reduction FROM the known NPC problem TO your problem. Step 4: Conclude: since the known NPC problem reduces to it, your problem is NPC.

第1步：证明该问题属于NP（给出多项式时间验证器）。第2步：选择一个已知NPC问题（如3-SAT、Vertex Cover）。第3步：构造从已知NPC问题到该问题的多项式时间归约。

第4步：结论：既然已知NPC问题可归约到它，该问题就是NPC。

Section 9: Longest Common Subsequence (LCS) / 最长公共子序列

Appears in 2/6 papers (MCQ + sometimes Part B). Fill the $c[i,j]$ table, then backtrack to find the actual LCS string.

2/6套试卷（选择题，有时也出大题）。填写 $c[i,j]$ 表格，然后回溯找出实际的LCS字符串。

Recurrence:

$c[i, j] = 0$ if $i = 0$ or $j = 0$

$c[i, j] = c[i-1, j-1] + 1$ if $x_i = y_j$ (match!)

$c[i, j] = \max(c[i-1, j], c[i, j-1])$ if $x_i \neq y_j$ (no match)

LCS Table Example: X = AATGTT, Y = AGCT

LCS表格示例：X = AATGTT, Y = AGCT

		A	G	C	T
	0	0	0	0	0
A	0	1	1	1	1
A	0	1	1	1	1
T	0	1	1	1	2
G	0	1	2	2	2
T	0	1	2	2	3

LCS length = $c[6,4] = 3$. Possible LCS: AGT (from backtracking).

LCS长度 = $c[6,4] = 3$ 。可能的LCS：AGT（通过回溯得到）。

Section 10: Horspool's Algorithm / Horspool算法（移位表）

Appears in 2/6 papers (MCQ + CW2). Build the shift table for a given pattern, then apply Horspool to count comparisons in a text match.

2/6套试卷（选择题+CW2）。给pattern建移位表，然后应用Horspool算法计算在文本中匹配时的比较次数。

Shift Table Rule: For a pattern of length m : $t(c) = m$ for all characters NOT in the pattern (default) $t(\text{pattern}[i]) = m - 1 - i$ for $i = 0..m-2$ (last char is NOT updated) In the exam: list $t(A)$, $t(G)$, $t(C)$, $t(T)$ explicitly.

移位表规则：对于长度为 m 的pattern： $t(c) = m$ 对于所有不在pattern中的字符（默认值） $t(\text{pattern}[i]) = m - 1 - i$ 对于 $i = 0..m-2$ （最后一个字符不更新） 考试中：明确列出 $t(A)$ 、 $t(G)$ 、 $t(C)$ 、 $t(T)$ 的值。

Example: pattern = DCCDADDC ($m=8$)

示例: pattern = DCCDADDC ($m=8$)

Char	Position(s) in pattern	$t(\text{char}) = m-1-\text{pos}$
D	0, 3, 6 (last=6, ignored)	$\min(8-1-0, 8-1-3) = \min(7, 4) = 4$
C	1	$8-1-1 = 6$
A	4	$8-1-4 = 4$
B (not in pattern)	-	8 (default)

Section 11: Sequence Alignment (DP) / 序列比对 (2023-24新增)

New in 2023-24 (CW2). Two types: GLOBAL alignment (Needleman-Wunsch) and LOCAL alignment (Smith-Waterman). Given a scoring matrix and gap penalty d .

2023-24新增（CW2）。两类：全局比对（Needleman-Wunsch）和局部比对（Smith-Waterman）。给出打分矩阵和gap penalty d 。

Global Alignment Recurrence (Needleman-Wunsch):

```
V[i,j] = max{
V[i-1, j-1] + s(x_i, y_j), (match/mismatch)
V[i-1, j] + d, (gap in Y)
V[i, j-1] + d (gap in X)
}
```

Local Alignment Recurrence (Smith-Waterman) - DIFFERENT from global:

```
V[i,j] = max{
V[i-1, j-1] + s(x_i, y_j),
V[i-1, j] + d,
V[i, j-1] + d,
0 <-- STOP at 0!
}
```

Backtrack from the MAXIMUM value in the table; stop when you hit 0.

DNA Scoring Matrix Example (CW2 2023-24):

DNA打分矩阵示例 (CW2 2023-24) :

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

Gap penalty d is NEGATIVE (e.g. $d = -5$). Match = positive score; mismatch = negative score.

Gap penalty d 是负数 (如 $d = -5$)。匹配=正数; 不匹配=负数。

Section 12: Coin Row Problem / 金币行问题 (DP)

Appears in 2/6 papers as Part B. You cannot pick two adjacent coins. Use DP to maximise the sum.

2/6套试卷出大题。不能选相邻的两枚硬币。用DP最大化总金额。

Recurrence:

$$F(n) = \max\{F(n-1), F(n-2) + c_n\} \text{ for } n \geq 2$$

$$F(0) = 0$$

$$F(1) = c_1$$

Meaning: either skip coin n ($F(n-1)$), or take coin n (cannot take $n-1$, so $F(n-2) + c_n$).

Example: coins = [5, 1, 2, 10, 6, 2]

示例: 硬币 = [5, 1, 2, 10, 6, 2]

n	0	1	2	3	4	5	6
c_n	-	5	1	2	10	6	2
$F(n)$	0	5	5	7	15	15	17

Optimal selection: coins at positions 1, 4, 6 -> $5 + 10 + 2 = 17$

最优选择: 位置1、4、6的硬币 -> $5 + 10 + 2 = 17$

Section 13: Other Low-Frequency Topics / 其他低频考点

These appear 0-1 times across all papers. Know the basic idea; do NOT prioritise for initial revision.

这些内容在所有试卷中出现0-1次。了解基本思想即可; 复习初期不必优先关注。

Topic	Exam Appearance	What to Know
Floyd-Warshall	2/6 (MCQ)	Shortest path between ALL pairs; recurrence: $D[i,j] = \min(D[i,j], D[i,k] + D[k,j])$
Bellman-Ford	1/6 (CW2)	Handles negative weights; relax all edges $ V -1$ times; detect negative cycle
Assembly-Line Scheduling	1/6 (CW1)	DP: $f1[j] = \min(f1[j-1] + a1,j, f2[j-1] + t2,j-1 + a1,j)$; backtrack for path
Change-Making (DP)	1/6 (Final 2020-21)	Recurrence: $F(n) = \min\{F(n-d1), F(n-d2), \dots\} + 1$

Merge comparisons	1/6 (Resit MCQ)	Merging two sorted arrays: at most $m+n-1$ comparisons
Euler Circuit	3/6 (MCQ)	ALL vertices have even degree $\rightarrow$ Euler circuit exists

Final Revision Checklist / 最后复习清单

Use this checklist to track your revision progress. Mark each item when you have mastered it.

用此清单跟踪复习进度。掌握每项后打勾。

- ☐ Time Complexity
- ☐ Big-O definition (memorise)
- ☐ Growth rate ranking (memorise order)
- ☐ Code $\rightarrow$ $O()$ identification (practice 10+ examples)
- ☐ BFS / DFS
- ☐ BFS visit order (practice 3+ graphs)
- ☐ DFS visit order (practice 3+ graphs)
- ☐ Difference between BFS and DFS (memorise table)
- ☐ MST (Kruskal & Prim)
- ☐ Kruskal: edge selection order
- ☐ Prim: vertex selection order (from given start vertex)
- ☐ MST properties (TRUE/FALSE - memorise all 6 statements)
- ☐ Dijkstra
- ☐ Label format: vertex(distance, predecessor)
- ☐ Step-by-step table (practice 2+ graphs)
- ☐ Dijkstra tree $\neq$ MST (know why)
- ☐ Sorting (Bubble & Selection)
- ☐ Bubble Sort: comparisons = $n(n-1)/2$, swaps vary
- ☐ Selection Sort: comparisons = $n(n-1)/2$, swaps $\leq n-1$
- ☐ Worked example: $A = [6,1,2,3,4,5]$
- ☐ Divide & Conquer (Part B)
- ☐ Describe technique (3 marks template)
- ☐ Design algorithm (pseudocode)
- ☐ Recurrence relation $T(n)$ (memorise common patterns)
- ☐ Complexity proof (iterative method OR substitution/induction)
- ☐ 0/1 Knapsack DP (Part B)
- ☐ Recurrence $V[i,j]$ (memorise)
- ☐ Fill table (practice 2+ examples)
- ☐ Backtracking to find optimal subset (practice 2+ examples)
- ☐ P / NP / NPC (Part B)
- ☐ Define P (memorise exact wording)

- [] Define NP (memorise exact wording)
- [] Define NPC (memorise exact wording)
- [] Polynomial-time reduction (explain + template)
- [] Prove a problem is NPC (4-step template)
- [] LCS
- [] Recurrence $c[i,j]$ (memorise)
- [] Fill table (practice 1+ example)
- [] Backtrack to find LCS string
- [] Horspool
- [] Build shift table (practice 1+ example)
- [] Simulate matching process
- [] Sequence Alignment (NEW)
- [] Global alignment recurrence (Needleman-Wunsch)
- [] Local alignment recurrence (Smith-Waterman) - note the '0'!
- [] Scoring matrix + gap penalty
- [] Coin Row
- [] Recurrence $F(n)$ (memorise)
- [] Manual calculation (practice 1+ example)

FINAL EXAM TIP: Do MCQs (Part A) in 30 minutes. Save 90 minutes for Part B (show ALL steps, even if the answer seems obvious).

期末考试提示：选择题（Part A）用30分钟完成。留90分钟给Part B（展示所有步骤，即使答案看起来很明显）。

IMPORTANT: In Part B, WRITING ALL STEPS is worth more marks than the final answer! Even if you know the answer, write the recurrence, the table, and the backtracking explicitly.

重要：在Part B中，写出所有步骤比最终答案值更多分！即使你知道答案，也要明确写出递推关系、表格和回溯过程。

[MCQ Quick Facts] Time Complexity / 时间复杂度

- Big-O: $f(n)=O(g(n))$ means exists $c>0, n_0$ such that $f(n)\leq c\cdot g(n)$ for all $n\geq n_0$
 → Big-O定义：存在正数 c 和 n_0 ，对所有 $n\geq n_0$ 有 $f(n)\leq c\cdot g(n)$
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$
 → 常见复杂度排序：常数<对数<线性<线性对数<平方<立方<指数<阶乘
- Sum of first n integers = $n(n+1)/2$, so nested loop $i=1..n, j=1..i$ gives $O(n^2)$
 → 前 n 项和= $n(n+1)/2$ ，双重循环 $i=1..n, j=1..i$ 为 $O(n^2)$

□ Theta notation requires BOTH upper and lower bound: $f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$

→ Theta记号要求上下界同时成立, 即 $f = O(g)$ 且 $f = \Omega(g)$

□ $\Omega(g(n))$ is the LOWER bound: exists $c > 0, n_0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

→ Omega是下界: 存在 $c > 0, n_0$ 使得 $f(n) \geq c \cdot g(n)$

[MCQ Quick Facts] BFS & DFS / 图遍历

□ BFS uses QUEUE (FIFO); DFS uses STACK (LIFO) — this is the #1 most tested fact

→ BFS用队列 (先进先出), DFS用栈 (后进先出) — 最常考的知识点

□ BFS finds shortest path in UNWEIGHTED graphs; DFS does NOT guarantee shortest path

→ BFS在无权图中找最短路径, DFS不能保证最短路径

□ Euler circuit exists iff all vertices have EVEN degree and graph is connected

→ 欧拉回路存在当且仅当所有顶点度数为偶数且图连通

□ Hamiltonian path visits each vertex exactly once; NP-complete to decide

→ 哈密顿路径恰好访问每个顶点一次; 判定问题是NP完全的

□ Adjacency matrix: $O(V^2)$ space, $O(1)$ edge lookup; Adjacency list: $O(V+E)$ space, $O(\text{degree})$ lookup

→ 邻接矩阵 $O(V^2)$ 空间 $O(1)$ 查边; 邻接表 $O(V+E)$ 空间 $O(\text{度数})$ 查边

[MCQ Quick Facts] MST / 最小生成树

□ Kruskal: sort edges by weight, add cheapest that does NOT form cycle. Uses Union-Find.

→ Kruskal: 按权排序边, 加入不形成环的最小边。用并查集。

□ Prim: grow tree from one vertex, always add cheapest edge TO the tree.

→ Prim: 从一个顶点生长, 始终加连向树的最小权边。

□ MST is NOT unique if edges have duplicate weights

→ 如果边权有重复, MST不唯一

□ Cut property: the lightest edge crossing a cut belongs to SOME MST

→ 切割性质: 跨越切割的最小权边一定属于某棵MST

□ Cycle property: the heaviest edge on a cycle is NOT in any MST

→ 环性质: 环上最重边一定不在任何MST中

[MCQ Quick Facts] Dijkstra / 最短路径

□ Dijkstra works only with NON-NEGATIVE weights; fails with negative edges

→ Dijkstra只适用于非负权边; 有负权边会出错

□ Dijkstra time: $O(V^2)$ with array, $O((V+E)\log V)$ with min-heap

→ Dijkstra时间: 数组 $O(V^2)$, 最小堆 $O((V+E)\log V)$

□ Dijkstra does NOT necessarily find longest path (that is NP-hard)

→ Dijkstra不能找最长路径 (最长路径是NP难的)

□ If all edges have equal weight, Dijkstra reduces to BFS

→ 所有边等权时Dijkstra退化为BFS

[MCQ Quick Facts] Sorting / 排序

□ Bubble sort: worst/avg $O(n^2)$, best $O(n)$ if already sorted; STABLE sort

→ 冒泡排序: 最坏/平均 $O(n^2)$, 已排序时最优 $O(n)$; 稳定排序

□ Selection sort: ALWAYS $O(n^2)$ comparisons, $n-1$ swaps; NOT stable

→ 选择排序: 始终 $O(n^2)$ 比较, $n-1$ 次交换; 不稳定

□ Merge sort: $O(n \log n)$ always, STABLE, but needs $O(n)$ extra space

→ 归并排序: 始终 $O(n \log n)$, 稳定, 但需 $O(n)$ 额外空间

□ Bubble sort swap count = number of INVERSIONS in input

→ 冒泡排序交换次数 = 输入中逆序对的数量

□ Selection sort does exactly $n-1$ swaps regardless of input order

→ 选择排序无论输入顺序如何都恰好做 $n-1$ 次交换

[MCQ Quick Facts] Divide & Conquer / 分治法

□ Divide & Conquer: divide into subproblems, solve recursively, combine results

→ 分治法: 分解子问题→递归求解→合并结果

□ Master theorem: $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b(a)}$

→ 主定理: $T(n) = aT(n/b) + f(n)$, 比较 $f(n)$ 与 $n^{\log_b(a)}$

□ Binary search: $T(n) = T(n/2) + O(1)$, so $T(n) = O(\log n)$

→ 二分查找: $T(n) = T(n/2) + O(1)$, 所以 $T(n) = O(\log n)$

□ Merge sort recurrence: $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$ by Master theorem case 2

→ 归并排序递推: $T(n) = 2T(n/2) + O(n)$, 主定理case2得 $O(n \log n)$

[MCQ Quick Facts] 0/1 Knapsack / 0-1背包

□ 0/1 Knapsack: each item either taken or NOT; fractional knapsack uses greedy (not DP)

→ 0/1背包: 每件物品取或不取; 分数背包用贪心不用DP

□ 0/1 Knapsack DP: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$

→ 0/1背包DP: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$

□ Time $O(nW)$, Space $O(nW)$ or $O(W)$ with rolling array

→ 时间 $O(nW)$, 空间 $O(nW)$ 或滚动数组 $O(W)$

□ Fractional knapsack is solved GREEDILY by value/weight ratio — NOT DP

→ 分数背包按性价比贪心求解——不用DP

□ To reconstruct which items are taken, backtrack from $dp[n][W]$

→ 回溯找选了哪些物品：从 $dp[n][W]$ 往回推

[MCQ Quick Facts] P/NP/NPC / 计算复杂性

□ P: solvable in polynomial time; NP: verifiable in polynomial time

→ P类：多项式时间可解；NP类：多项式时间可验证

□ NP does NOT mean 'Non-Polynomial'! It means 'Non-deterministic Polynomial'

→ NP不是'非多项式'！而是'非确定性多项式'

□ NP-complete: in NP AND every NP problem reduces to it in polynomial time

→ NP完全：属于NP且所有NP问题可多项式归约到它

□ Examples of NP-complete: SAT, 3-SAT, Hamiltonian Cycle, TSP, 0/1 Knapsack, Subset Sum

→ NP完全问题举例：SAT、3-SAT、哈密顿回路、TSP、0/1背包、子集和

□ Examples of P: sorting, MST, shortest path, LCS, primality testing

→ P问题举例：排序、MST、最短路径、LCS、素性测试

□ To prove a problem is NP-complete: (1) show it is in NP, (2) reduce a known NP-complete problem to it

→ 证明NP完全：(1)证在NP中，(2)将已知NP完全问题归约到它

[MCQ Quick Facts] LCS / 最长公共子序列

□ LCS DP: if $X[i]=Y[j]$, $dp[i][j]=dp[i-1][j-1]+1$; else $dp[i][j]=\max(dp[i-1][j], dp[i][j-1])$

→ LCS递推：若 $X[i]=Y[j]$ 则 $dp[i][j]=dp[i-1][j-1]+1$ ，否则取左/上最大值

□ LCS length table is $(m+1) \times (n+1)$, first row and first column are all 0

→ LCS表大小 $(m+1) \times (n+1)$ ，首行首列全为0

□ Time $O(mn)$, Space $O(mn)$; can reduce space to $O(\min(m,n))$

→ 时间 $O(mn)$ 空间 $O(mn)$ ；空间可优化到 $O(\min(m,n))$

□ LCS is NOT the same as Longest Common Substring (which requires consecutive)

→ LCS不是最长公共子串（子串要求连续）

[MCQ Quick Facts] Horspool / 字符串匹配

□ Horspool: preprocess pattern to build shift table; $shift = \max(1, m - \text{last_occurrence}[\text{text_char}])$

→ Horspool：预处理模式串建移位表；移位 $=\max(1, m-\text{最后出现位置}[\text{text_char}])$

□ If text char not in pattern, $shift = m$ (pattern length)

→ 如果文本字符不在模式串中，移位=模式串长度 m

□ Last character of pattern has shift 0 in table (but actual shift ≥ 1)

→ 模式串最后一个字符在表中移位为0 (但实际移位 ≥ 1)

□ Best case $O(n/m)$, worst case $O(nm)$

→ 最优 $O(n/m)$, 最坏 $O(nm)$

[MCQ Quick Facts] Sequence Alignment / 序列比对

□ Global alignment (Needleman-Wunsch): align entire sequences; Local alignment (Smith-Waterman): find best matching substring

→ 全局比对Needleman-Wunsch: 比对完整序列; 局部比对Smith-Waterman: 找最佳匹配子段

□ Gap penalty can be linear (d per gap) or affine (opening + extension cost)

→ 空位罚分可以是线性 (每个空位d) 或仿射 (开空位+扩展罚分)

□ Scoring matrix: match=+score, mismatch=-score; choice of scores affects alignment

→ 打分矩阵: 匹配+分, 错配-分; 分数选择影响比对结果

[MCQ Quick Facts] Coin Row & Change-Making / 硬币问题

□ Coin Row: $dp[i] = \max(dp[i-1], dp[i-2] + \text{coin}[i])$; cannot take adjacent coins

→ 硬币行: $dp[i] = \max(dp[i-1], dp[i-2] + \text{coin}[i])$; 不能取相邻硬币

□ Base cases: $dp[0] = 0$, $dp[1] = \text{coin}[1]$

→ 边界: $dp[0] = 0$, $dp[1] = \text{coin}[1]$

□ Change-making: $dp[j] = \min(dp[j - \text{coin}_i] + 1)$ for all $\text{coin}_i \leq j$; finds min number of coins

→ 找零问题: $dp[j] = \min(dp[j - \text{coin}_i] + 1)$, 找最少硬币数

□ Change-making is NOT the same as Coin Row (different constraints)

→ 找零问题与硬币行问题不同 (约束不同)

Appendix: Mock MCQ Practice / 模拟选择题

15道高频模拟题, 每题含答案+英中双语解析。 / 15 high-frequency mock questions with bilingual explanations.

Q1. What is the time complexity of the following code?

for i in range(n): for j in range(i, n): print(i,j)

A. $O(n)$

B. $O(n \log n)$

C. $O(n^2)$

D. $O(n^3)$

Answer: C — Inner loop runs (n-i) times for each i, total = $n + (n-1) + \dots + 1 = n(n+1)/2 = O(n^2)$

→ 内层循环每次执行(n-i)次, 总和= $n + (n-1) + \dots + 1 = n(n+1)/2 = O(n^2)$

Q2. Which data structure does BFS use?

- A. Stack
- B. Queue
- C. Priority Queue
- D. Hash Table

Answer: B — BFS uses a Queue (FIFO). DFS uses a Stack (LIFO).

→ BFS用队列（先进先出），DFS用栈（后进先出）

Q3. In Kruskal's algorithm, what data structure is used to detect cycles?

- A. Stack
- B. Queue
- C. Union-Find (Disjoint Set)
- D. Binary Search Tree

Answer: C — Kruskal uses Union-Find to efficiently check if adding an edge creates a cycle.

→ Kruskal用并查集高效检测加入边是否形成环

Q4. Dijkstra's algorithm fails when the graph has:

- A. Positive weights
- B. Negative weights
- C. Zero weights
- D. Self-loops

Answer: B — Dijkstra assumes all edges are non-negative. Negative edges can cause incorrect results.

→ Dijkstra假设边权非负，负权边会导致结果不正确

Q5. How many swaps does Selection Sort perform on an array of n elements?

- A. $O(n \log n)$
- B. $O(n^2)$
- C. Exactly $n-1$
- D. Depends on input order

Answer: C — Selection sort always makes exactly $n-1$ swaps: it swaps the minimum to position i for each i .

→ 选择排序始终做 $n-1$ 次交换：每轮把最小值交换到位置 i

Q6. Which of the following is a P problem?

- A. 3-SAT
- B. Hamiltonian Cycle
- C. Finding MST
- D. 0/1 Knapsack Decision

Answer: C — MST can be found in $O(E \log V)$ — polynomial time. The others are NP-complete.

→ MST可在 $O(E \log V)$ 多项式时间内找到，其余是NP完全问题

Q7. The recurrence $T(n) = 2T(n/2) + O(n)$ solves to:

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(n^2)$
- D. $O(\log n)$

Answer: B — By Master Theorem: $a=2, b=2, f(n)=O(n)=O(n^{\log_2(2)})=O(n^1)$, case 2 $\Rightarrow O(n \log n)$

→ 主定理: $a=2, b=2, f(n)=O(n)=O(n^1)$, 属于case2, 解为 $O(n \log n)$

Q8. In the 0/1 Knapsack DP table, $dp[i][w]$ represents:

- A. Maximum value using all items with capacity w
- B. Maximum value using first i items with capacity w
- C. Weight of item i
- D. Number of items selected

Answer: B — $dp[i][w]$ = max value achievable using only the first i items with knapsack capacity w .

→ $dp[i][w]$ =只用前 i 件物品、容量为 w 时能获得的最大价值

Q9. In Horspool's algorithm, if the last character of the current text window does NOT appear in the pattern, the shift is:

- A. 0
- B. 1
- C. m (pattern length)
- D. $m-1$

Answer: C — If the character is not in the pattern at all, we can safely shift the pattern by its full length m .

→ 如果字符不在模式串中, 可以安全地移动模式串整个长度 m

Q10. Which is true about NP-complete problems?

- A. They can be solved in polynomial time
- B. If any NP-complete problem is in P, then $P=NP$
- C. They are not in NP
- D. They can only be solved by brute force

Answer: B — This is the fundamental property: if any NP-complete problem has a polynomial algorithm, then $P=NP$.

→ 核心性质: 如果任何一个NP完全问题有多项式算法, 则 $P=NP$

Q11. In LCS, if $X[i] \neq Y[j]$, then $dp[i][j] =$

- A. $dp[i-1][j-1]$
- B. $\max(dp[i-1][j], dp[i][j-1])$
- C. $dp[i-1][j-1]+1$
- D. 0

Answer: B — When characters don't match, take the maximum of skipping $X[i]$ or skipping $Y[j]$.

→ 字符不匹配时, 取跳过 $X[i]$ 或跳过 $Y[j]$ 的最大值

Q12. An Euler circuit exists in a connected graph if and only if:

- A. All vertices have odd degree
- B. All vertices have even degree
- C. The graph is a tree
- D. The graph has exactly 2 vertices of odd degree

Answer: B — Euler circuit requires ALL vertices to have even degree. If exactly 2 have odd degree, Euler PATH exists (not circuit).

→ 欧拉回路要求所有顶点度数为偶数。恰好2个奇数度则有欧拉路径（非回路）

Q13. Bubble sort on a REVERSE-sorted array of n elements makes how many comparisons?

- A. $n-1$
- B. $n(n-1)/2$
- C. $n \log n$
- D. n^2

Answer: B — Reverse-sorted is worst case: comparisons = $(n-1)+(n-2)+\dots+1 = n(n-1)/2$.

→ 逆序是最坏情况：比较次数= $(n-1)+(n-2)+\dots+1=n(n-1)/2$

Q14. The Floyd-Warshall algorithm computes:

- A. Minimum Spanning Tree
- B. Single-source shortest path
- C. All-pairs shortest path
- D. Maximum flow

Answer: C — Floyd-Warshall finds shortest paths between ALL pairs of vertices in $O(V^3)$.

→ Floyd-Warshall在 $O(V^3)$ 内找到所有顶点对之间的最短路径

Q15. In the Coin Row problem, $dp[i]$ represents:

- A. Maximum value of coin i
- B. Maximum sum picking from first i coins with no two adjacent
- C. Minimum coins needed
- D. Total value of all coins

Answer: B — $dp[i] = \max(dp[i-1], dp[i-2]+coin[i])$: best sum from first i coins, no two adjacent selected.

→ $dp[i]=\max(dp[i-1],dp[i-2]+coin[i])$: 前 i 个硬币中不取相邻的最大和

Q16. A graph has 5 vertices with degrees 3,3,2,2,2. Does it have an Euler circuit?

- A. Yes, because the graph is connected
- B. No, because not all degrees are even
- C. Yes, because all degrees are even
- D. Cannot determine from degrees alone

Answer: B — Euler circuit requires ALL vertices to have EVEN degree. Here vertices with degree 3 are odd.

→ 欧拉回路要求所有顶点度数为偶数。这里有度数为3的奇数度顶点，所以不存在欧拉回路

Q17. What is the time complexity of DFS on a graph with V vertices and E edges using adjacency list?

- A. $O(V)$
- B. $O(V+E)$
- C. $O(V^2)$
- D. $O(VE)$

Answer: B — DFS visits each vertex once and examines each edge once with adjacency list, giving $O(V+E)$.

→ DFS用邻接表每个顶点访问一次、每条边检查一次，时间 $O(V+E)$

Q18. In Prim's algorithm, starting from vertex A , the next edge added is always:

- A. The cheapest edge in the entire graph
- B. The cheapest edge connected to the current tree
- C. The cheapest edge not yet considered
- D. A random edge from the remaining edges

Answer: B — Prim always adds the minimum-weight edge that connects a tree vertex to a non-tree vertex.

→ Prim始终加连接当前树内顶点和树外顶点的最小权边

Q19. Merge sort performs how many comparisons in the BEST case?

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(n^2)$
- D. $O(\log n)$

Answer: B — Merge sort always does $O(n \log n)$ comparisons regardless of input order — that's its advantage.

→ 归并排序无论输入如何都做 $O(n \log n)$ 次比较——这是它的优点

Q20. Which sorting algorithm has the SAME worst-case and best-case time complexity?

- A. Bubble Sort
- B. Selection Sort
- C. Insertion Sort
- D. Quick Sort

Answer: B — Selection sort always does $O(n^2)$ comparisons in all cases. Only the number of swaps varies.

→ 选择排序所有情况都是 $O(n^2)$ 比较。只有交换次数可能不同

Q21. In the 0/1 Knapsack problem with $n=4$ items and capacity $W=10$, the DP table size is:

- A. 4×10
- B. 5×11
- C. $n \times W$

D. 4×11

Answer: B — DP table is $(n+1) \times (W+1) = 5 \times 11$, because we need rows $0..n$ and columns $0..W$ for base cases.

→ DP表大小 $(n+1) \times (W+1) = 5 \times 11$, 因为需要 $0..n$ 行和 $0..W$ 列的边界条件

Q22. What happens if we apply Dijkstra to a graph with a negative-weight edge?

A. It still finds the correct shortest path

B. It may produce incorrect results

C. It will crash

D. It automatically switches to Bellman-Ford

Answer: B — Dijkstra's greedy choice fails with negative edges — it may settle a vertex prematurely with a non-optimal distance.

→ Dijkstra的贪心策略在负权边上会失败——可能过早确定一个非最优距离的顶点

Q23. The Bellman-Ford algorithm can handle graphs with negative edges and detects:

A. Euler circuits

B. Negative-weight cycles

C. Hamiltonian paths

D. Minimum cuts

Answer: B — Bellman-Ford detects negative-weight cycles by running one extra iteration — if any distance updates, a negative cycle exists.

→ Bellman-Ford通过多跑一轮检测负权环——如果距离还能更新则存在负权环

Q24. In the LCS of $X=ABCB DAB$ and $Y=BDCAB$, the length of LCS is:

A. 3

B. 4

C. 5

D. 6

Answer: B — One LCS is BCAB with length 4. Build the DP table to verify.

→ 一个LCS是BCAB, 长度4。可建DP表验证

Q25. In Horspool's algorithm with pattern SEARCH, the shift for character 'S' is:

A. 0

B. 5

C. 4

D. 6

Answer: C — Pattern SEARCH has length 6. S appears at position 0 (0-indexed). $\text{Shift} = m-1-0 = 5$? No: S also appears at position 3. Use rightmost occurrence: $\text{shift} = m-1-3 = 2$? Wait — last char is H at pos 5, we compute for the text char aligned with pos 5. For 'S': rightmost S (excluding last pos) is at index 3. $\text{Shift} = m-1-3 = 2$. But S is also at index 0. Rightmost non-last = index 3, $\text{shift}=2$. Hmm, let me reconsider: for pattern SEARCH, the shift table: $S \rightarrow m-1-3=2$, $E \rightarrow m-1-1=4$, $A \rightarrow m-1-2=3$, $R \rightarrow m-1-4=1$, $C \rightarrow m-1-0=5$? No, C is at index 0? No: $S(0)E(1)A(2)R(3)C(4)H(5)$. For S: rightmost S not at last position = index 0, $\text{shift} = 6-1-0 = 5$. Wait

S is at index 0 only. Shift for $S = m-1-0 = 5$.

→ 模式SEARCH长度6, S只出现在位置0 (排除末位), 移位= $m-1-0=5$

Q26. Which problem is solved by finding the longest path in a DAG?

- A. Shortest path
- B. Activity selection (critical path)
- C. Minimum spanning tree
- D. Maximum flow

Answer: B — Longest path in a DAG models the critical path in project scheduling. In general graphs, longest path is NP-hard, but in DAGs it can be solved by negating weights and running shortest path.

→ DAG中的最长路径对应项目调度中的关键路径。一般图最长路径是NP难的, 但DAG可解

Q27. The number of edges in a spanning tree of a connected graph with V vertices is:

- A. V
- B. V-1
- C. V^2
- D. $V(V-1)/2$

Answer: B — A spanning tree on V vertices always has exactly V-1 edges. This is a fundamental graph property.

→ V个顶点的生成树始终恰好有V-1条边, 这是图的基本性质

Q28. Which of the following is TRUE about the fractional knapsack problem?

- A. It requires dynamic programming
- B. It can be solved greedily by value-to-weight ratio
- C. It is NP-complete
- D. It has the same solution as 0/1 knapsack

Answer: B — Fractional knapsack is solved by greedy: sort items by value/weight ratio, take as much as possible of the best ratio item first.

→ 分数背包用贪心求解: 按性价比排序, 优先取性价比最高的物品

Mock Written Questions / 模拟大题

2道高频模拟大题, 含完整解题步骤和英中双语解析。

Written Q1: 0/1 Knapsack DP Table Construction (12 marks)

Given 4 items with weights and values:

Item 1: $w=2, v=3$ Item 2: $w=3, v=4$ Item 3: $w=4, v=5$ Item 4: $w=5, v=6$

Knapsack capacity $W = 8$. Fill in the COMPLETE DP table and identify which items are selected.

给定4件物品的重量和价值: 物品1: $w=2, v=3$ 物品2: $w=3, v=4$ 物品3: $w=4, v=5$

物品4: $w=5, v=6$ 。背包容量 $W=8$ 。填写完整DP表并指出选了哪些物品。

Solution / 解答:

Step 1: Create DP table $dp[i][w]$ for $i=0..4$, $w=0..8$

第一步：建立 $dp[i][w]$ 表， $i=0..4$, $w=0..8$

Step 2: Initialize $dp[0][w] = 0$ for all w (0 items = 0 value)

第二步：初始化 $dp[0][w]=0$ （没有物品时价值为0）

Step 3: For each item i , for each capacity w :

If $w_i \leq w$: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$

Else: $dp[i][w] = dp[i-1][w]$

第三步：对每件物品 i 和每个容量 w ：若 $w_i \leq w$ 则 $dp[i][w]=\max(dp[i-1][w], dp[i-1][w-w_i]+v_i)$ ，否则 $dp[i][w]=dp[i-1][w]$

Complete DP Table / 完整DP表:

$i \backslash w$	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1(w_2, v_3)	0	0	3	3	3	3	3	3	3
2(w_3, v_4)	0	0	3	4	4	7	7	7	7
3(w_4, v_5)	0	0	3	4	5	7	8	9	9
4(w_5, v_6)	0	0	3	4	5	7	8	9	10

Step 4: Maximum value = $dp[4][8] = 10$

第四步：最大价值 = $dp[4][8] = 10$

Step 5: Backtrack to find selected items:

第五步：回溯找选中的物品：

- $dp[4][8]=10$, $dp[3][8]=9$, so Item 4 IS selected ($10 \neq 9$). Go to $dp[3][8-5]=dp[3][3]=4$
- $dp[3][3]=4$, $dp[2][3]=4$, so Item 3 NOT selected ($4 = 4$). Stay at $dp[2][3]$
- $dp[2][3]=4$, $dp[1][3]=3$, so Item 2 IS selected ($4 \neq 3$). Go to $dp[1][3-3]=dp[1][0]=0$
- $dp[1][0]=0$, $dp[0][0]=0$, so Item 1 NOT selected

Selected: Items 2 and 4. Total weight = $3+5 = 8$. Total value = $4+6 = 10$.

选中：物品2和物品4。总重量= $3+5=8$ ，总价值= $4+6=10$ 。

Written Q2: Divide and Conquer - Find a Peak Element (10 marks)

Given array $A = [1, 3, 7, 5, 2, 4, 6, 8, 3]$. A peak is an element $A[i]$ such that $A[i] \geq A[i-1]$ and $A[i] \geq A[i+1]$. Use the divide-and-conquer algorithm to find a peak. Show ALL steps.

给定数组 $A=[1,3,7,5,2,4,6,8,3]$ 。峰值是满足 $A[i] \geq A[i-1]$ 且 $A[i] \geq A[i+1]$ 的元素。用分治法找一个峰值，写出所有步骤。

Solution / 解答:

Algorithm: binary search on the array. Compare middle with neighbors, go to the side with the larger neighbor.

算法：在数组上做二分搜索。比较中间元素与邻居，走向更大邻居的一侧。

Step 1: $A = [1, 3, 7, 5, 2, 4, 6, 8, 3]$, $mid = \text{index } 4$, $A[4] = 2$

$A[3]=5 > A[4]=2$, so go LEFT. Search $[1, 3, 7, 5, 2]$ (indices 0-4)

第1步: $\text{mid}=\text{索引}4$, $A[4]=2$ 。 $A[3]=5>2$, 向左搜索 $[0..4]$

Step 2: Subarray $[1, 3, 7, 5, 2]$, $\text{mid} = \text{index } 2$, $A[2] = 7$

$A[1]=3 < A[2]=7$ and $A[3]=5 < A[2]=7$. $A[2] = 7$ is a PEAK!

第2步: $\text{mid}=\text{索引}2$, $A[2]=7$ 。 $A[1]=3<7$ 且 $A[3]=5<7$ 。 $A[2]=7$ 是一个峰值!

Verification: $A[2]=7$, $A[1]=3 \leq 7$, $A[3]=5 \leq 7$. Confirmed peak.

验证: $A[2]=7$, 左边 $3 \leq 7$, 右边 $5 \leq 7$ 。确认为峰值。

Time complexity: $T(n) = T(n/2) + O(1)$, by Master Theorem $T(n) = O(\log n)$

时间复杂度: $T(n)=T(n/2)+O(1)$, 主定理得 $T(n)=O(\log n)$

Note: $A[7]=8$ is also a peak ($A[6]=6 < 8$, $A[8]=3 < 8$), but the algorithm finds the first one it encounters.

注意: $A[7]=8$ 也是峰值($A[6]=6<8$, $A[8]=3<8$), 但算法找到的是第一个遇到的峰值。